

Politechnika Łódzka



IAPŁ

INSTYTUT AUTOMATYKI
POLITECHNIKI ŁÓDZKIEJ

Zakład sterowania robotów

ROBOTY MOBILNE

Instrukcja laboratoryjna 1

Kinematyka robotów mobilnych



Spis treści

1	Spis oznaczeń	2
2	Transformacje	3
2.1	Opis układów współrzędnych	3
2.2	Wektor stanu robota	3
2.3	Transformacje pomiędzy układami odniesienia	4
2.3.1	Transformacja z układu lokalnego do globalnego	4
2.3.2	Transformacja z układu globalnego do lokalnego	4
3	Ograniczenia kinematyczne	5
3.1	Założenia upraszczające	5
3.2	Koło stałe (<i>Fixed standard wheel</i>)	5
3.2.1	Warunek czystego toczenia (<i>pure rolling constraint</i>)	6
3.2.2	Warunek braku poślizgu bocznego (<i>sliding constraint</i>)	6
3.3	Koło skrętne (<i>Steered standard wheel</i>)	6
3.3.1	Warunek czystego toczenia	7
3.3.2	Warunek braku poślizgu bocznego	7
3.4	Koło omni (<i>Omnidirectional wheel</i>)	7
3.4.1	Warunek czystego toczenia	7
3.4.2	Warunek braku poślizgu bocznego	7
4	Złożenie ograniczeń wszystkich kół	8
4.1	Ogólna postać macierzowa ograniczeń	8
4.2	Macierze równań kinematycznych	8
5	Kinematyka robota	9
5.1	Kinematyka prosta	9
5.2	Kinematyka odwrotna	9
6	Dlaczego używamy pseudoinwersji macierzy	9
6.1	Wprowadzenie	9
6.2	Idea pseudoinwersji Moore’a–Penrose’a	10
6.3	Interpretacja fizyczna	10
6.4	Przykłady dla różnych typów robotów	10
6.4.1	Mecanum	10
6.4.2	Car-like (Ackermann)	11
6.4.3	Skid-steer	12
7	Obliczanie kąta skrętu koła (<i>steering angle</i>)	14
7.1	Czysta translacja	14

7.2	Czysty obrót	14
7.3	Połączenie translacji i obrotu	15
7.4	Korekta kąta dla ruchu wstecznego	15
8	Analiza mobilności i sterowalności robota mobilnego	15
8.1	Stopnie swobody robota (<i>Degrees of Freedom, DoF</i>)	15
8.2	Mobilność robota (<i>Degree of Mobility, δ_m</i>)	16
8.3	Sterowalność robota (<i>Degree of Steerability, δ_s</i>)	16
8.4	Manewrowalność robota (<i>Degree of Maneuverability, δ_M</i>)	17
9	Program laboratorium - ćwiczenia	18

1 Spis oznaczeń

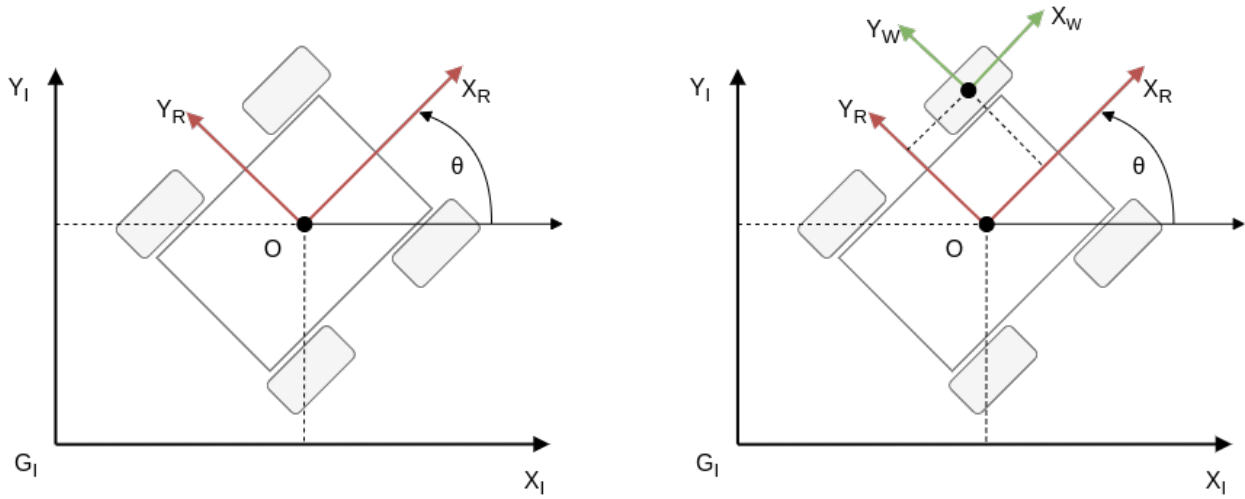
- x - współrzędna x robota w układzie globalnym
- y - współrzędna y robota w układzie globalnym
- θ - orientacja robota w układzie globalnym
- $\mathbf{R}(Z, \theta)$ - macierz rotacji podstawowej wokół osi Z
- $\dot{\xi}_{\mathbf{I}}$ - wektor prędkości robota w układzie globalnym
- $\dot{\xi}_{\mathbf{R}}$ - wektor prędkości robota w układzie lokalnym robota
- r - promień koła
- $\dot{\varphi}$ - prędkość obrotowa koła ($\frac{rad}{s}$)
- l - odległość koła od wybranego punktu zaczepienia układu robota na korpusie
- α - orientacja koła względem środka robota, liczony od osi X robota
- β - kąt skręcenia koła w układzie robota
- β_s - kąt skręcenia koła w układzie robota koła skrętnego
- φ_{roller} - prędkość obrotowa pasywnego rollera
- r_{roller} - promień pasywnego rollera
- γ - kąt ustawienia rollera względem osi X koła
- $\mathbf{A}$ - macierz ograniczeń kinematycznych wynikających z warunku toczenia i braku poślizgu
- $\mathbf{B}$ - macierz przekształcająca prędkości kół na prędkość robota
- $\mathbf{J}_{1f}$ - ograniczenia **czystego toczenia** dla kół stałych
- $\mathbf{J}_{1s}(\beta_s)$ - ograniczenia **czystego toczenia** dla kół skrętnych z podstawionym znanym kątem skrętu β_s dla danego koła
- $\mathbf{C}_{1f}$ - ograniczenia **braku poślizgu poprzecznego** dla kół stałych
- $\mathbf{C}_{1s}(\beta_s)$ - ograniczenia **braku poślizgu poprzecznego** dla kół skrętnych z podstawionym znanym kątem skrętu β_s dla danego koła
- $\mathbf{J}_2 = \text{diag}(r_1, r_2, \dots, r_n)$ - macierz diagonalna zawierająca promienie poszczególnych kół
- $\mathbf{M}^*$ - pseudoinwersję Moore'a-Penrose'a macierzy $\mathbf{M}$
- δ_m - stopień mobilności robota mobilnego
- δ_s - stopień sterowalności robota mobilnego
- δ_M - stopień manewrowalności robota mobilnego

2 Transformacje

Budowę modelu kinematycznego robota mobilnego należy rozpocząć od zrozumienia wpływu każdego z kół na jego zdolność poruszania się. Każde koło w inny sposób przenosi ruch obrotowy na ruch całej platformy. Zależy to od jego rodzaju, położenia względem środka robota oraz parametrów geometrycznych (m.in. promienia i orientacji) czy typu koła. Każde z kół wprowadza do modelu określone ograniczenia kinematyczne, a ich superpozycja definiuje mobilność całego robota.

Na zajęciach analizowany jest ruch robota na płaszczyźnie, dlatego jego pozycja jest opisana trzema stopniami swobody: położeniem (x, y) oraz orientacją θ względem osi Z .

2.1 Opis układów współrzędnych



Rys. 2.1: Graficzne przedstawienie robota na płaszczyźnie z zaznaczeniem układów współrzędnych związanych z podłożem, robotem i pojedynczym kołem.

Układ $O : \{X_R, Y_R\}$ jest układem lokalnym związanym z platformą robota. Zwrot osi X_R odpowiada kierunkowi jazdy do przodu, natomiast początek układu znajduje się zazwyczaj pośrodku w punkcie pomiędzy kołami napędowymi. Punkt ten jest punktem referencyjnym przy określaniu pozycji poszczególnych kół.

Aby opisać położenie robota w przestrzeni, jego lokalny układ współrzędnych należy odnieść do globalnego (inercjalnego) układu $G : \{X_I, Y_I\}$. Operacja ta polega na rzutowaniu układu robota do układu globalnego.

2.2 Wektor stanu robota

Pozycję robota w układzie globalnym (inercjalnym) można przedstawić za pomocą wektora stanu:

$$\xi_I = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \quad (2.1)$$

Wektor ten opisuje translacyjne i rotacyjne położenie robota w dwuwymiarowej przestrzeni roboczej.

2.3 Transformacje pomiędzy układami odniesienia

2.3.1 Transformacja z układu lokalnego do globalnego

W celu przejścia pomiędzy układem lokalnym robota O a inercyjnym układem odniesienia G stosuje się macierz rotacji wokół osi Z Rys. 2.2.

$$\mathbf{R}(Z, \theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.2)$$

gdzie θ oznacza orientację robota względem osi Z_I układu globalnego.

Macierz ta opisuje obrót wektora o kąt θ wokół osi Z w **prawoskrętnym układzie współrzędnych** i jest przykładem **macierzy obrotu podstawowego**, znanej z kursu *Podstaw Robotyki*.

Równanie 2.3 umożliwia wyznaczenie prędkości robota w układzie globalnym (G) na podstawie jego prędkości w układzie lokalnym (O). Transformacja ta ma postać:

$$\dot{\xi}_I = \mathbf{R}(Z, \theta) \dot{\xi}_R \quad (2.3)$$

2.3.2 Transformacja z układu globalnego do lokalnego

Odwrotne przekształcenie, czyli przejście z inercyjnego układu odniesienia G do układu lokalnego robota O , umożliwia analizę ruchu w odniesieniu do robota. W tym przypadku wykorzystuje się transpozycję macierzy rotacji podstawowej $R(Z, \theta)$, oznaczoną jako $\mathbf{R}^T(Z, \theta)$, ponieważ macierz rotacji jest **ortogonalna**:

$$\mathbf{R}^{-1}(Z, \theta) = \mathbf{R}^T(Z, \theta)$$

Macierz $\mathbf{R}^T(Z, \theta)$ zachowuje tę samą strukturę co $\mathbf{R}(Z, \theta)$, jednak zamienia miejscami wartości sinusów, zmieniając ich znak:

$$\mathbf{R}^T(Z, \theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.4)$$

Transformacja ta umożliwia przeliczenie wektorów prędkości z układu globalnego na lokalny i ma postać:

$$\dot{\xi}_R = \mathbf{R}^T(\theta) \dot{\xi}_I \quad (2.5)$$

Równanie 2.5 pozwala wyznaczyć prędkości robota w jego własnym układzie odniesienia (O), mając znane prędkości w układzie globalnym (G).

3 Ograniczenia kinematyczne

Podczas tworzenia modelu kinematycznego robota mobilnego należy uwzględnić ograniczenia, jakie wprowadzają jego koła. Każde koło, w zależności od konstrukcji i umiejscowienia, ogranicza możliwe kierunki ruchu platformy.

3.1 Założenia upraszczające

Aby możliwe było sformułowanie ogólnych równań ruchu, w celu uproszczenia rozważań należy dokonać dwóch założeń:

— **Brak poślizgu bocznego (*no-sliding constraint*)**

Koło toczy się po powierzchni w taki sposób, że w punkcie styku koła i podłoża występuje dokładnie jeden punkt kontaktu i w tym punkcie nie występuje poślizg poprzeczny. Wektor prędkości postępowej koła (zaczepiony w punkcie obrotu koła, np. oś) jest zawsze skierowany równolegle do podłoża.

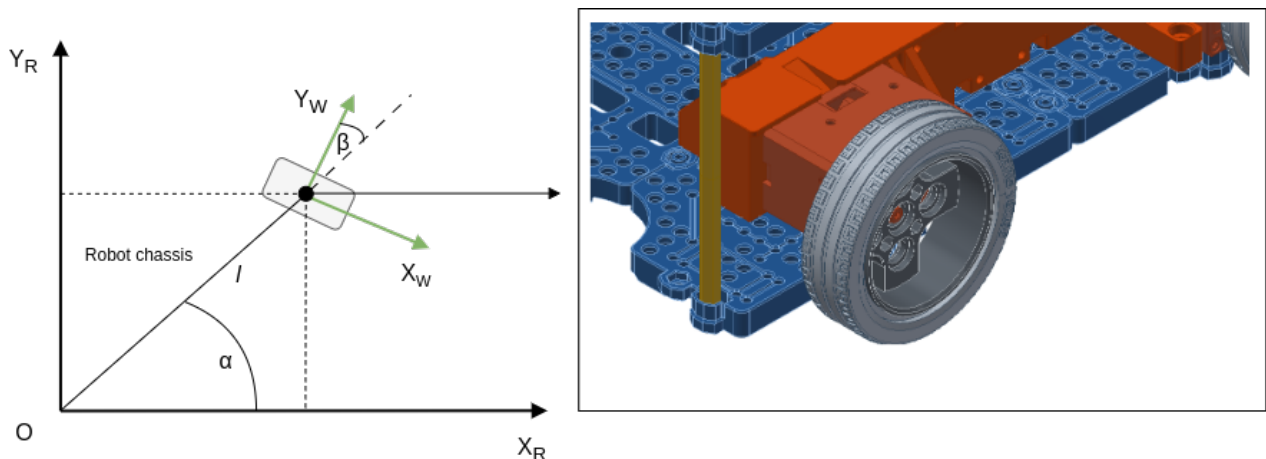
— **Czyste toczenie (*rolling without slipping*)**

Koło obraca się bez poślizgu wzdłużnego. Inaczej mówiąc - gdy następuje ruch robota koło musi się obracać w odpowiednim kierunku. (*Rolling constraint*)

Złożenie ograniczeń poszczególnych kół pozwala stworzyć model kinematyczny całego pojazdu. Poniżej omówiono trzy najczęściej spotykane typy kół w robotyce mobilnej.

3.2 Koło stałe (*Fixed standard wheel*)

Koło stałe jest sztywno zamocowane do korpusu robota i nie posiada możliwości skrętu. **Posiada jeden stopień swobody – może toczyć się tylko wzdłuż swojej osi głównej.** Oznacza to, że kierunek ruchu jest ściśle określony przez orientację osi koła. Koła te są często stosowane w robotyce mobilnej jako stabilizatory lub elementy wspomagające, zapewniając prostoliniowy ruch bez możliwości zmiany kierunku.



Rys. 3.1: Koło stałe i jego orientacja w układzie robota.

3.2.1 Warunek czystego toczenia (*pure rolling constraint*)

$$\begin{bmatrix} \sin(\alpha + \beta) & -\cos(\alpha + \beta) & -l \cos(\beta) \end{bmatrix} \dot{\xi}_{\mathbf{R}} - r \dot{\varphi} = 0 \quad (3.1)$$

gdzie:

- $\dot{\varphi}$ - prędkość obrotowa koła ($\frac{rad}{s}$)
- r - promień koła
- l - odległość koła od wybranego punktu zaczepienia układu robota na korpusie.

Wektor trójelementowy w nawiasach określa, jak składowe prędkości robota $(\dot{x}, \dot{y}, \dot{\theta})$ wpływają na prędkość wzdłuż płaszczyzny toczenia koła. Ten ruch zgodnie z założeniem czystego toczenia musi być równy $\dot{\varphi}r$ czyli dokładnie tyle ile wynika z obrotu koła.

3.2.2 Warunek braku poślizgu bocznego (*sliding constraint*)

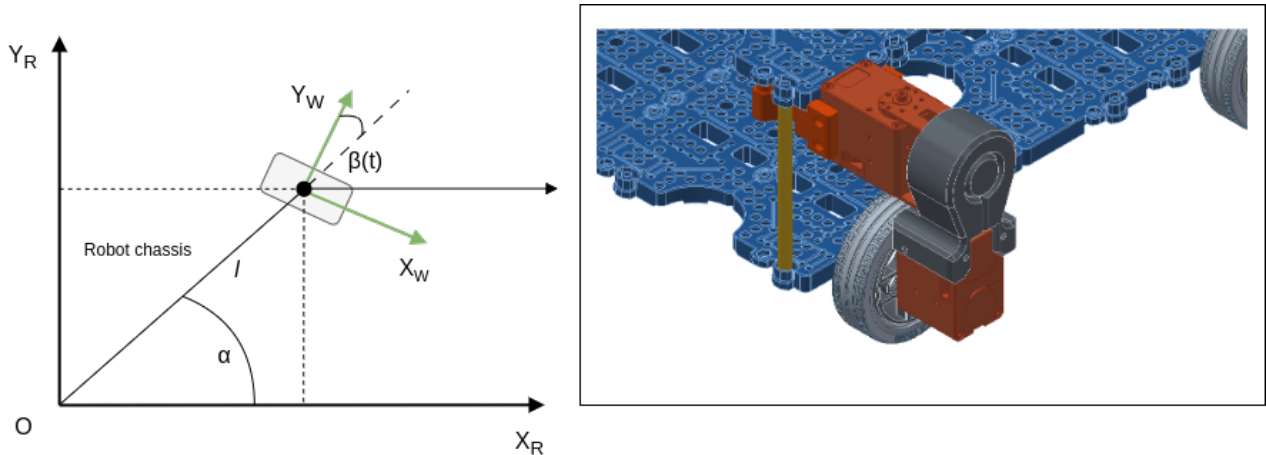
Warunek na brak poślizgu poprzecznego zapewnia, że ruch koła prostopadły do jego płaszczyzny wynosi zero (nie ma innego ruchu w punkcie styku koła z podłożem niż toczenie).

$$\begin{bmatrix} \cos(\alpha + \beta) & \sin(\alpha + \beta) & l \sin(\beta) \end{bmatrix} \dot{\xi}_{\mathbf{R}} = 0 \quad (3.2)$$

Oznacza to, że ruch robota w kierunku prostopadłym do płaszczyzny koła jest zabroniony — w punkcie styku nie występuje poślizg poprzeczny.

3.3 Koło skrętne (*Steered standard wheel*)

Koło skrętne ma dwa stopnie swobody: obrót wokół własnej osi (toczenie, ruch wzdłuż linii prostej) **oraz zmianę kierunku** (skręt). Dzięki możliwości sterowania kątem skrętu, koło to pozwala na elastyczne poruszanie się robota w różnych kierunkach. Koła skrętne są szeroko stosowane w pojazdach i robotach mobilnych, gdzie wymagana jest precyzyjna kontrola nad kierunkiem ruchu. Koła tego typu są stosowane m.in. w robotach typu Ackermann.



Rys. 3.2: Koło skrętne w układzie robota.

3.3.1 Warunek czystego toczenia

$$\begin{bmatrix} \sin(\alpha + \beta) & -\cos(\alpha + \beta) & -l \cos(\beta) \end{bmatrix} \dot{\xi}_{\mathbf{R}} - r\dot{\varphi} = 0 \quad (3.3)$$

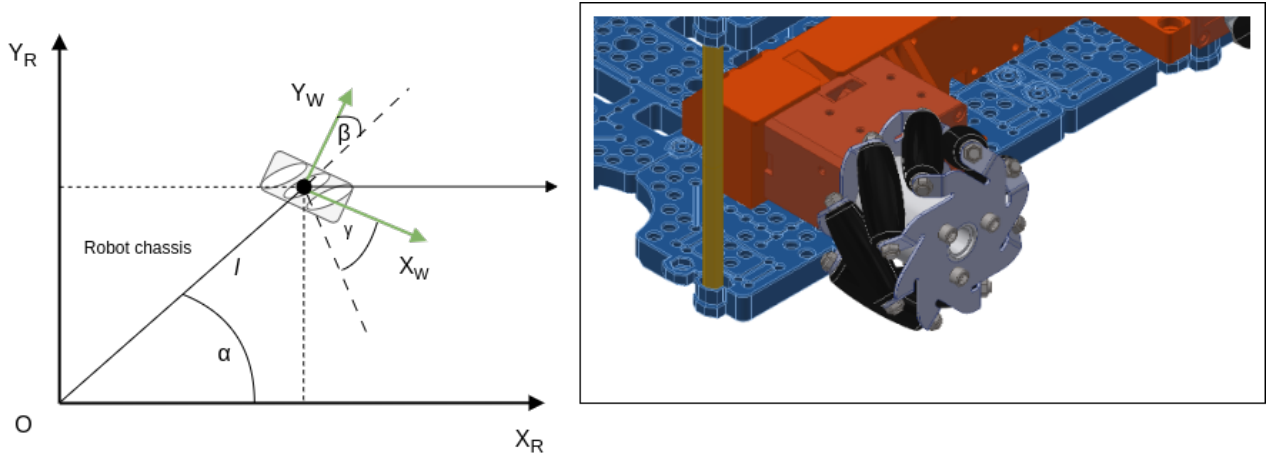
3.3.2 Warunek braku poślizgu bocznego

$$\begin{bmatrix} \cos(\alpha + \beta) & \sin(\alpha + \beta) & l \sin(\beta) \end{bmatrix} \dot{\xi}_{\mathbf{R}} = 0 \quad (3.4)$$

3.4 Koło omni (*Omnidirectional wheel*)

Koło omni posiada zestaw pasywnych rolek rozmieszczonych wokół obwodu, dzięki czemu umożliwia ruch w dowolnym kierunku bez konieczności zmiany orientacji. Dzięki tym dodatkowym rolkom, koło omni eliminuje ograniczenie poślizgu poprzecznego, umożliwiając ruch w dowolnym kierunku przy zachowaniu kontaktu z podłożem.

Takie rozwiązanie nadaje robotowi trzy stopnie swobody w płaszczyźnie: ruch wzdłuż osi X , Y oraz obrót wokół osi Z . Koła tego typu są często wykorzystywane w robotyce mobilnej, gdzie wymagana jest duża zwrotność i możliwość ruchu w wielu kierunkach bez skręcania całego robota.



Rys. 3.3: Koło omnidirectional (omni) w układzie robota.

3.4.1 Warunek czystego toczenia

$$\begin{bmatrix} \sin(\alpha + \beta + \gamma) & -\cos(\alpha + \beta + \gamma) & (-l)\cos(\beta + \gamma) \end{bmatrix} \dot{\xi}_{\mathbf{R}} - r\dot{\varphi}\cos(\gamma) = 0 \quad (3.5)$$

3.4.2 Warunek braku poślizgu bocznego

$$\begin{bmatrix} \cos(\alpha + \beta + \gamma) & \sin(\alpha + \beta + \gamma) & l\sin(\beta + \gamma) \end{bmatrix} \dot{\xi}_{\mathbf{R}} - r\dot{\varphi}\sin(\gamma) - r_{roller}\dot{\varphi}_{roller} = 0 \quad (3.6)$$

gdzie:

- φ_{roller} – prędkość obrotowa pasywnego rollera,
- r_{roller} – promień pasywnego rollera,
- γ – kąt ustawienia rollera względem osi X koła.

4 Złożenie ograniczeń wszystkich kół

Aby opisać pełną kinematykę robota mobilnego, konieczne jest połączenie ograniczeń wynikających z każdego koła w jeden spójny model matematyczny. Każde koło – w zależności od tego, czy jest stałe, skrętne lub omnidirectional – wnosi własny zestaw równań ograniczających dopuszczalne kierunki ruchu.

4.1 Ogólna postać macierzowa ograniczeń

Dla robota posiadającego n kół (stałych lub skrętnych) wszystkie ograniczenia można zapisać w postaci macierzowej:

$$\mathbf{A}\dot{\xi}_R = \mathbf{B}\dot{\varphi} \quad (4.1)$$

Gdzie:

- $\mathbf{A}$ – macierz ograniczeń kinematycznych wynikających z warunku toczenia i braku poślizgu,
- $\mathbf{B}$ – macierz przekształcająca prędkości kół na prędkość robota.

Ograniczenia na brak poślizgu wprowadzają jedynie koła **stałe** i **skrętne**, ponieważ mogą one toczyć się wyłącznie wzdłuż osi X swojego lokalnego układu. Wektor $\dot{\xi}_R$ opisuje prędkość robota w układzie lokalnym, natomiast $\dot{\varphi}$ to wektor prędkości kątowych poszczególnych kół.

4.2 Macierze równań kinematycznych

$$\mathbf{A} = \begin{bmatrix} \mathbf{J}_{1f} \\ \mathbf{J}_{1s}(\beta_s) \\ \mathbf{C}_{1f} \\ \mathbf{C}_{1s}(\beta_s) \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} \mathbf{J}_2 \\ \mathbf{0} \end{bmatrix} \quad (4.2)$$

gdzie poszczególne elementy macierzy oznaczają:

- $\mathbf{J}_{1f}$ – ograniczenia **czystego toczenia** dla kół stałych,
- $\mathbf{J}_{1s}(\beta_s)$ – ograniczenia **czystego toczenia** dla kół skrętnych z podstawionym znanym kątem skrętu β_s dla danego koła,
- $\mathbf{C}_{1f}$ – ograniczenia **braku poślizgu poprzecznego** dla kół stałych,
- $\mathbf{C}_{1s}(\beta_s)$ – ograniczenia **braku poślizgu poprzecznego** dla kół skrętnych z podstawionym znanym kątem skrętu β_s dla danego koła
- $\mathbf{J}_2 = \text{diag}(r_1, r_2, \dots, r_n)$ – macierz diagonalna zawierająca promienie poszczególnych kół.

5 Kinematyka robota

Kinematyka robota mobilnego opisuje zależność pomiędzy prędkościami jego kół a ruchami całej platformy. Na podstawie wcześniej zdefiniowanych ograniczeń kinematycznych można wyznaczyć dwa podstawowe modele: **kinematykę prostą** (forward kinematics) oraz **kinematykę odwrotną** (inverse kinematics). Oba modele są ważne dla sterowania, symulacji i lokalizacji robota.

5.1 Kinematyka prosta

Czyli: w jaki sposób zmierzona prędkość kół przekłada się na prędkość robota?

Kinematyka prosta pozwala określić, jak obrót kół wpływa na ruch całej platformy w układzie robota. W tym modelu znane są prędkości kątowe kół $\dot{\phi}$, a celem jest wyznaczenie wektora prędkości robota $\dot{\xi}_R$

$$\dot{\xi}_R = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{B} \dot{\phi} \quad (5.1)$$

5.2 Kinematyka odwrotna

Czyli w jaki sposób zadana prędkość robota przekłada się na prędkości kół? Kinematyka odwrotna stanowi proces odwrotny do kinematyki prostej. Znając żądane prędkości postępowe w kierunku XY i kątowe robota $\dot{\xi}_R$, chcemy obliczyć, z jakimi prędkościami powinny obracać się jego koła $\dot{\phi}$, aby osiągnąć ten ruch.

$$\dot{\phi} = (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T \mathbf{A} \dot{\xi}_R \quad (5.2)$$

6 Dlaczego używamy pseudoinwersji macierzy

6.1 Wprowadzenie

Podczas analizy kinematyki różnych typów robotów mobilnych okazuje się, że układy równań opisujące zależność między prędkościami kół a ruchem robota mają różną strukturę. W niektórych przypadkach można obliczyć dokładne rozwiązanie, a w innych — jedynie rozwiązanie przybliżone.

Dzieje się tak, ponieważ macierze opisujące te układy ($\mathbf{A}$, $\mathbf{B}$) mogą mieć różne wymiary i rząd. W szczególności, jeśli liczba równań jest większa od liczby niewiadomych (układ **nadokreślony**) lub mniejsza (układ **niedookreślony**), klasyczne odwrócenie macierzy nie jest możliwe.

Właśnie wtedy stosuje się pseudoinwersję Moore’a–Penrose’a macierzy np. $\mathbf{M}$, oznaczaną jako $\mathbf{M}^+$.

6.2 Idea pseudoinwersji Moore’a–Penrose’a

Niech układ równań kinematycznych ma postać:

$$\mathbf{A} \dot{\xi}_{\mathbf{R}} = \mathbf{B} \dot{\phi} \quad (6.1)$$

Jeśli $\mathbf{B}$ jest macierzą prostokątną (niekwadratową), nie można obliczyć jej klasycznej odwrotności $\mathbf{B}^{-1}$. Zamiast tego szukamy rozwiązania, które minimalizuje błąd:

$$\min_{\dot{\phi}} \|\mathbf{A} \dot{\xi}_{\mathbf{R}} - \mathbf{B} \dot{\phi}\|^2 \quad (6.2)$$

Rozwiązanie tego problemu w sensie najmniejszych kwadratów ma postać:

$$\dot{\phi} = (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T \mathbf{A} \dot{\xi}_{\mathbf{R}} \quad (6.3)$$

lub ogólniej:

$$\dot{\phi} = \mathbf{B}^+ \mathbf{A} \dot{\xi}_{\mathbf{R}} \quad (6.4)$$

gdzie $\mathbf{B}^+$ to pseudoinwersja Moore’a–Penrose’a macierzy $\mathbf{B}$:

$$\mathbf{B}^+ = (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T \quad (6.5)$$

6.3 Interpretacja fizyczna

W kontekście robotów mobilnych pseudoinwersja oznacza, że nie zawsze da się spełnić wszystkie ograniczenia kinematyczne dokładnie. Z tego powodu wyznaczamy takie prędkości kół $\dot{\phi}$, które **najlepiej przybliżają** zadany ruch robota $\dot{\xi}_{\mathbf{R}}$ w sensie najmniejszych błędów kwadratowych.

W ten sposób pseudoinwersja stanowi matematyczny „kompromis” między wszystkimi ograniczeniami ruchu, szczególnie gdy występują poślizgi lub ograniczenia nie mogą być idealnie spełnione.

6.4 Przykłady dla różnych typów robotów

6.4.1 Mecanum

Układ z kołami Mecanum może poruszać się z poprzeczną prędkością v_y , wobec tego nie spełni ograniczeń na brak poślizgu bocznego – tych równań nie uwzględniamy w macierzy $\mathbf{A}$.

Tab. 6.1: Parametry kół robota Mecanum

Koło	Położenie	Rodzaj	Ogr. toczenia	Ogr. poślizgu
1	lewy przód	mecanum	tak	nie
2	lewy tył	mecanum	tak	nie
3	prawy tył	mecanum	tak	nie
4	prawy przód	mecanum	tak	nie

$$\mathbf{A} = \begin{bmatrix} \sin(\alpha_1 + \beta_1 + \gamma_1) & -\cos(\alpha_1 + \beta_1 + \gamma_1) & -l_1 \cos(\beta_1 + \gamma_1) \\ \sin(\alpha_2 + \beta_2 + \gamma_2) & -\cos(\alpha_2 + \beta_2 + \gamma_2) & -l_2 \cos(\beta_2 + \gamma_2) \\ \sin(\alpha_3 + \beta_3 + \gamma_3) & -\cos(\alpha_3 + \beta_3 + \gamma_3) & -l_3 \cos(\beta_3 + \gamma_3) \\ \sin(\alpha_4 + \beta_4 + \gamma_4) & -\cos(\alpha_4 + \beta_4 + \gamma_4) & -l_4 \cos(\beta_4 + \gamma_4) \end{bmatrix} \quad (6.6)$$

gdzie:

- α_n – orientacja koła n w układzie robota,
- β_n – kąt skrętu koła,
- γ_n – orientacja rolki na kole Mecanum,
- l_n – odległość koła od środka układu robota.

$$\mathbf{B} = \begin{bmatrix} r \cos(\gamma_1) & 0 & 0 & 0 \\ 0 & r \cos(\gamma_2) & 0 & 0 \\ 0 & 0 & r \cos(\gamma_3) & 0 \\ 0 & 0 & 0 & r \cos(\gamma_4) \end{bmatrix} \quad (6.7)$$

gdzie:

- r – promień koła,
- γ_n – orientacja rolki na kole Mecanum.

Przy wyliczaniu wektora prędkości kół ($\dot{\phi}$) rozwiązujemy układ czterech równań w celu wyliczenia czterech niewiadomych wartości prędkości. Możemy zastosować wzór:

$$\dot{\phi} = \mathbf{B}^{-1} \mathbf{A} \dot{\xi}_{\mathbf{R}} \quad (6.8)$$

ponieważ macierz $\mathbf{B}$ jest macierzą kwadratową oraz ma niezerowe wartości na diagonalu (promienie kół), więc jest macierzą pełnego rzędu, a tym samym odwracalną. W tym przypadku nie ma potrzeby stosowania pseudoinwersji.

6.4.2 Car-like (Ackermann)

Układ z Car-like, jak sama nazwa wskazuje, jest podobny do konfiguracji kół w samochodzie. Układ Ackermannia polega na takim sterowaniu skrętem kół podczas jazdy po łuku, aby każde koło poruszało się po okręgu o wspólnym środku (tzw. *ICR* – chwilowy punkt obrotu). Gwarantuje to ruch bez poślizgu.

Tab. 6.2: Parametry kół robota Car-like

Koło	Położenie	Rodzaj	Ogr. toczenia	Ogr. poślizgu
1	lewy przód	skrętne	tak	tak
2	lewy tył	stałe	tak	tak
3	prawy tył	stałe	tak	tak
4	prawy przód	skrętne	tak	tak

$$\mathbf{A} = \begin{bmatrix} \sin(\alpha_1 + \beta_1 + \theta_1) & -\cos(\alpha_1 + \beta_1 + \theta_1) & -l_1 \cos(\beta_1 + \theta_1) \\ \sin(\alpha_2 + \beta_2) & -\cos(\alpha_2 + \beta_2) & -l_2 \cos(\beta_2) \\ \sin(\alpha_3 + \beta_3) & -\cos(\alpha_3 + \beta_3) & -l_3 \cos(\beta_3) \\ \sin(\alpha_4 + \beta_4 + \theta_4) & -\cos(\alpha_4 + \beta_4 + \theta_4) & -l_4 \cos(\beta_4 + \theta_4) \\ \cos(\alpha_1 + \beta_1 + \theta_1) & \sin(\alpha_1 + \beta_1 + \theta_1) & l_1 \sin(\beta_1 + \theta_1) \\ \cos(\alpha_2 + \beta_2) & \sin(\alpha_2 + \beta_2) & l_2 \sin(\beta_2) \\ \cos(\alpha_3 + \beta_3) & \sin(\alpha_3 + \beta_3) & l_3 \sin(\beta_3) \\ \cos(\alpha_4 + \beta_4 + \theta_4) & \sin(\alpha_4 + \beta_4 + \theta_4) & l_4 \sin(\beta_4 + \theta_4) \end{bmatrix} \quad (6.9)$$

gdzie:

- α_n – orientacja koła n w układzie robota,
- β_n – kąt skrętu koła,
- θ_n – obliczony kąt skrętu,
- l_n – odległość koła od środka układu robota.

$$\mathbf{B} = \begin{bmatrix} r & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & r & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & r & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & r & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (6.10)$$

gdzie:

- r – promień koła.

Dla robota o geometrii samochodu układ jest **nadokreślony**. Zawiera więcej równań (ograniczeń) niż niezależnych stopni swobody ruchu. Zatem należy zastosować pseudoinwersję **Moore’a–Penrose’a** macierzy $\mathbf{B}$.

$$\dot{\phi} = (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T \mathbf{A} \dot{\xi}_{\mathbf{R}} = \mathbf{B}^+ \mathbf{A} \dot{\xi}_{\mathbf{R}} \quad (6.11)$$

6.4.3 Skid-steer

W tym układzie wszystkie koła są stałe. Realizacja skrętu zawsze wiąże się z poślizgiem bocznym kół, przez co warunki na brak poślizgu naturalnie nie mogą być spełnione (ale są wymagane, więc muszą być uwzględnione w macierzy $\mathbf{A}$).

Tab. 6.3: Parametry kół robota Skid-steer

Koło	Położenie	Rodzaj	Ogr. toczenia	Ogr. poślizgu
1	lewy przód	stałe	tak	tak
2	lewy tył	stałe	tak	tak
3	prawy tył	stałe	tak	tak
4	prawy przód	stałe	tak	tak

$$\mathbf{A} = \begin{bmatrix} \sin(\alpha_1 + \beta_1) & -\cos(\alpha_1 + \beta_1) & -l_1 \cos(\beta_1) \\ \sin(\alpha_2 + \beta_2) & -\cos(\alpha_2 + \beta_2) & -l_2 \cos(\beta_2) \\ \sin(\alpha_3 + \beta_3) & -\cos(\alpha_3 + \beta_3) & -l_3 \cos(\beta_3) \\ \sin(\alpha_4 + \beta_4) & -\cos(\alpha_4 + \beta_4) & -l_4 \cos(\beta_4) \\ \cos(\alpha_1 + \beta_1) & \sin(\alpha_1 + \beta_1) & l_1 \sin(\beta_1) \\ \cos(\alpha_2 + \beta_2) & \sin(\alpha_2 + \beta_2) & l_2 \sin(\beta_2) \\ \cos(\alpha_3 + \beta_3) & \sin(\alpha_3 + \beta_3) & l_3 \sin(\beta_3) \\ \cos(\alpha_4 + \beta_4) & \sin(\alpha_4 + \beta_4) & l_4 \sin(\beta_4) \end{bmatrix} \quad (6.12)$$

gdzie:

- α_n – orientacja koła n w układzie robota,
- β_n – kąt skrętu koła,
- l_n – odległość koła od środka układu robota.

$$\mathbf{B} = \begin{bmatrix} r & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & r & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & r & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & r & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (6.13)$$

gdzie:

- r – promień koła.

W tym przypadku podobnie jak dla robota Car-like układ jest nadokreślony i stosujemy pseudoinwersję.

W robocie typu Car-like dzięki temu, że spełniamy wszystkie równania (tak dobieramy kąty skrętu kół, aby koła poruszały się po okręgach o wspólnym środku), pomimo wykorzystania metody najmniejszych kwadratów wynik będzie dokładny. W przypadku Skid-steer, gdzie część równań nie może być spełniona, wynik nie będzie dokładny, a jedynie przybliżony. Jednak w praktyce optymalne rozwiązanie takiego układu pozwala na osiągnięcie przez robota prędkości bliskich pożądanym.

Należy pamiętać, że w przypadku takich robotów, gdzie występuje poślizg, obliczenie odometrii (kinematyka prosta) będzie obciążone dużym i szybko narastającym błędem.

7 Obliczanie kąta skrętu koła (*steering angle*)

Kąt skrętu koła (θ_{steering}) określa kierunek, w którym powinno być ustawione koło, aby jego płaszczyzna toczenia była zgodna z rzeczywistym wektorem prędkości robota. Jego poprawne wyznaczenie jest kluczowe w robotach z kołami skrętnymi (np. Ackermann, Swerve Drive), gdzie orientacja kół wpływa bezpośrednio na możliwość jazdy po zakręcie bez poślizgu.

7.1 Czysta translacja

W przypadku czystej translacji (brak obrotu) zadana jest kombinacja prędkości w osiach X i Y . Kąt skrętu jest obliczany jako:

$$\theta = \text{atan2}(v_y, v_x) \quad (7.1)$$

gdzie:

- v_x – składowa prędkości wzdłuż osi X (ruch „do przodu”),
- v_y – składowa prędkości wzdłuż osi Y (ruch „na bok”).

Interpretacja:

- Jeśli robot porusza się idealnie do przodu ($v_y = 0$), to $\theta = 0$ — koła są ustawione równolegle do osi X_R . Inaczej zadany wektor prędkości pokrywa się z nominalnym kierunkiem toczenia kół
- Jeśli robot porusza się po skosie, np. z $v_x = v_y$, to $\theta = 45^\circ$.
- Wartość funkcji atan2 automatycznie uwzględnia odpowiedni ćwiartkowy znak kierunku ruchu.

7.2 Czysty obrót

Podczas czystego obrotu robot nie wykonuje ruchu postępowego ($v_x = v_y = 0$), a jedynie rotację wokół własnej osi z prędkością ω_z .

Każde z kół porusza się po okręgu wokół środka robota. Kierunek, w którym powinno być ustawione koło, jest prostopadły do promienia tego okręgu. Kąt skrętu można więc wyrazić jako:

$$\theta = -\beta \quad (7.2)$$

gdzie:

- β – to stały kąt skręcenia koła w układzie robota.

Interpretacja:

- Dla kół ustawionych na wprost ($\beta = 0$), ich oś toczenia jest styczna do okręgu ruchu — nie wymagają dodatkowego skrętu.
- Dla kół położonych pod kątem, np. z boku robota, skręt $\theta = -\beta$ zapewnia, że ich płaszczyzna toczenia jest zawsze styczna do toru ruchu.

7.3 Połączenie translacji i obrotu

W przypadku kombinacji translacji i obrotu (np. dla układów Ackermanna), kąt skrętu koła wyliczany jest na podstawie prędkości robota i odległości od środka obrotu:

$$\begin{cases} v_{x,wheel} = v_x - \omega_z \cdot y \\ v_{y,wheel} = v_y + \omega_z \cdot x \end{cases} \quad (7.3)$$

gdzie:

- v_x – prędkość liniowa w osi X ,
- v_y – prędkość liniowa w osi Y ,
- ω_z – prędkość kątowna wokół osi Z ,
- x – odległość środka robota w osi X ,
- y – odległość środka robota w osi Y .

Kąt skrętu obliczany jest jako:

$$\theta_{steering} = \text{atan2}(v_{y,wheel}, v_{x,wheel}) \quad (7.4)$$

7.4 Korekta kąta dla ruchu wstecznego

Podczas jazdy wstecz kierunek wektora prędkości robota odwraca się, co wpływa na wyznaczony kąt skrętu. Aby zachować spójność orientacji kół, wprowadza się korektę:

$$\text{if } (v_x < 0) \text{ then } \theta_{steering} + = \pm\pi \quad (7.5)$$

8 Analiza mobilności i sterowalności robota mobilnego

8.1 Stopnie swobody robota (*Degrees of Freedom, DoF*)

Stopnie swobody określają, jakie niezależne ruchy może wykonać robot w swojej przestrzeni operacyjnej. Dla robota mobilnego poruszającego się na płaszczyźnie 2D mogą istnieć trzy stopnie swobody:

- przemieszczenie wzdłuż osi X ,
- przemieszczenie wzdłuż osi Y ,
- obrót wokół osi Z (zmiana orientacji).

Nie każdy robot posiada możliwość kontrolowania wszystkich trzech komponentów ruchu.

8.2 Mobilność robota (*Degree of Mobility*, δ_m)

Mobilność robota określa, ile niezależnych stopni swobody jego ruchu można kontrolować bezpośrednio poprzez zmianę prędkości kół. Innymi słowy, informuje ona, w ilu kierunkach robot może się poruszać dzięki swoim napędom — bez dodatkowego sterowania kątem skrętu kół.

Mobilność definiuje się wzorem:

$$\delta_m = 3 - \text{rank}(\mathbf{C}_1(\beta_s)) \quad (8.1)$$

gdzie:

- $\mathbf{C}_1(\beta_s)$ – macierz ograniczeń bocznych wynikających z warunku braku poślizgu bocznego dla wszystkich kół.

Interpretacja wartości δ_m :

- $\delta_m = 3$ – pełna mobilność (robot może poruszać się w kierunku x , y i obracać wokół własnej osi Z),
- $\delta_m = 2$ – np. robot różnicowy, który nie może poruszać się bokiem,
- $\delta_m = 1$ – np. trójkołowiec z jednym napędzanym kołem,
- $\delta_m = 0$ – robot statyczny (brak możliwości ruchu).

Rząd macierzy $\text{rank}(\mathbf{C}_1(\beta_s))$ określa liczbę niezależnych ograniczeń (liczbę wektorów liniowo niezależnych w macierzy) nałożonych na ruch robota. Informuje, czy przekształcenie za pośrednictwem tej macierzy powoduje utratę wymiaru. Inaczej mówiąc, pokazuje, ile wymiarów ruchu zostaje zablokowanych przez warunki kinematyczne (np. brak poślizgu bocznego).

- Jeśli rząd macierzy $\mathbf{C}_1$ jest równy 3, to trójwymiarowy wektor prędkości zostaje przekształcony we wszystkich trzech wymiarach na inny wektor 3-elementowy. W tym przypadku wszystkie trzy stopnie swobody są ograniczone – robot nie może poruszać się w żadnym kierunku (brak mobilności).
- Jeśli rząd wynosi 2, oznacza to utratę rzędu macierzy co powoduje, że wektor 3-elementowy zostaje rzutowany na płaszczyznę. W tym przypadku jedno z możliwych przemieszczeń jest zablokowane – robot może się poruszać tylko w dwóch kierunkach (np. do przodu i obracać się).

Macierz $\mathbf{C}_1$ zawiera projekcje (rzuty) ruchów wszystkich kół na kierunki **prostopadłe do płaszczyzn ich toczenia** — czyli na kierunki, w których koła nie powinny się poruszać (poślizg boczny).

Wniosek: im większy rząd macierzy $\mathbf{C}_1$, tym więcej ograniczeń kinematycznych ma robot, a jego mobilność jest mniejsza.

8.3 Sterowalność robota (*Degree of Steerability*, δ_s)

Sterowalność robota określa, ile niezależnych kół skrętnych może aktywnie zmieniać kierunek ruchu robota poprzez zmianę orientacji osi toczących się. Sterowalność definiuje się jako:

$$\delta_s = \text{rank}(\mathbf{C}_{1s}(\beta_s)) \quad (8.2)$$

gdzie $\mathbf{C}_{1s}(\beta_s)$ jest macierzą ograniczeń na brak poślizgu dla kół skrętnych z podstawionym znanym kątem skrętu β_s dla danego koła.

Interpretacja:

- Zmiana kąta koła skrętnego **nie wpływa bezpośrednio na pozycję robota** — efekt tej zmiany jest widoczny dopiero **podczas ruchu**, gdy koło zaczyna się toczyć w nowym kierunku.
- Wzrost rzędu macierzy C_{1s} oznacza, że robot posiada **więcej niezależnych możliwości sterowania kierunkiem ruchu** $\Rightarrow$ **większą manewrowalność**.
- Warto zauważyć, że **to samo koło skrętne** może jednocześnie **zmniejszyć mobilność** (ponieważ chwilowo dodaje ograniczenie na ruch), ale **zwiększyć sterowalność** (bo pozwala na zmianę orientacji koła).

Typowe wartości sterowalności

- $\delta_s = 0$ – brak kół skrętnych (np. *differential drive*),
- $\delta_s = 1$ – np. trójkołowiec z jednym skrętnym kołem,
- $\delta_s = 2$ – np. robot z dwoma niezależnymi kołami skrętnymi.

8.4 Manewrowalność robota (*Degree of Maneuverability*, δ_M)

Całkowita liczba stopni swobody, które można kontrolować (prędkością lub sterowaniem), definiowana jest jako:

$$\delta_M = \delta_m + \delta_s \quad (8.3)$$

gdzie:

- (δ_m) – stopień mobilności robota,
- (δ_s) – stopień sterowalności robota,

Dwa roboty o tej samej manewrowalności δ_M nie muszą być równoważne pod względem zachowania.

Przykład porównawczy Roboty z napędem różnicowym (*differential drive*) i roboty trójkołowe (*tricycle*) mają taką samą manewrowalność:

$$\delta_M = 2$$

Jednak różnice są istotne:

- Dla napędu różnicowego:

$$\delta_m = 2, \quad \delta_s = 0 \quad \Rightarrow \quad \text{cała manewrowalność pochodzi z mobilności (napędu).}$$

- Dla trójkołowca:

$$\delta_m = 1, \quad \delta_s = 1 \quad \Rightarrow \quad \text{manewrowalność wynika częściowo z napędu, a częściowo z możliwości skrętu.}$$

Mimo tej samej liczby dostępnych stopni swobody (manewrowalności), sposób ich realizacji może być zupełnie inny.

9 Program laboratorium - ćwiczenia

Szczegółowy opis każdego z zadań od 1 do 4 (pliki `.ipynb` lub ich odpowiedniki w wersji `.pdf`) znajduje się w katalogu z zadaniami dla danej kinematyki robota lub w odpowiedniej sekcji na Wikampie.

W plikach zadaniowych `.ipynb` bezpośrednio **przed komórką do uzupełnienia** pojawia się na czerwono tekst w „Uzupełnij poniższą komórkę”. Pod nim może znajdować się bardziej rozbudowany opis ćwiczenia.

Ćwiczenie 1 - Parametry fizyczne robota

Dla robota mobilnego na danym stanowisku należy wyznaczyć parametry fizyczne robota. Dla każdego koła należy podać:

- (x, y) - współrzędne koła w układzie robota
- l - odległość koła od początku układu robota (zapisać funkcjami Python)
- α - orientację koła w układzie robota (zapisać funkcjami Python)
- β - kąt skrętu koła (zapisać funkcjami Python)

Dodatkowo dla robota **Mecanum** należy podać kąt rollera γ jako wartość liczbową.

Ćwiczenie 2 – Kinematyka odwrotna

Należy uzupełnić 2 komórki. Jedną z parametrami fizycznymi robota wyznaczonymi w ćwiczeniu 1 (wystarczy skopiować). Druga komórka dotyczy uzupełnienia funkcji `cmd_vel_callback`. Funkcja `cmd_vel_callback` odbiera wektor prędkości robota (w formacie `geometry_msgs/Twist`), przelicza je na prędkości każdego z kół i wysyła na temacie `/cmd_joint_states`.

Należy uzupełnić funkcję `cmd_vel_callback` aby realizowała funkcjonalność kinematyki odwrotnej robota mobilnego.

1. (Opcjonalnie) Obliczyć kąty skrętu kół θ , jeśli kinematyka robota tego wymaga.
2. Obliczyć prędkości kół $\dot{\phi}$ na podstawie wektora prędkości robota $\dot{\xi}_R$. Należy skorzystać ze wzoru:

$$\dot{\phi} = (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T \mathbf{A} \dot{\xi}_R \quad (9.1)$$

gdzie $\mathbf{A}$ jest macierzą ograniczeń kinematycznych a $\mathbf{B}$ jest macierzą transformacji prędkości kół.

3. Uzupełnić wiadomość `joint_state`.
4. Zweryfikować poprawność wyników na rzeczywistym robocie. Sprawdzić czy po wysłaniu sterowania z interfejsu użytkownika (*Jupyter Notebook*) robot porusza się w odpowiednim kierunku np. jeśli ma jechać do przodu to jedzie do przodu, a nie do tyłu albo wcale się nie porusza. Wystarczą testy polegające na obserwacji zachowania robota po zadaniu sterowania. Bez szczegółowych testów i zbierania danych.

Ćwiczenie 3 – Kinematyka prosta

Należy uzupełnić dwie komórki:

1. pierwszą – z parametrami fizycznymi robota wyznaczonymi w ćwiczeniu 1 (wystarczy je skopiować),
2. drugą – dotyczącą implementacji funkcji `joint_states_callback`.

Funkcja `joint_states_callback` odbiera wektor prędkości i kątów skrętu każdego koła (w formacie `geometry_msgs/JointState`), przelicza je na wypadkowe prędkości robota, a następnie na ich podstawie wyznacza odometrię, która jest publikowana na temacie `/odometry`.

Należy uzupełnić funkcję `joint_states_callback` tak, aby realizowała funkcjonalność kinematyki prostej.

1. **Wyznaczyć różnicę czasu δt** między kolejnymi wiadomościami przekazywanymi do `joint_states_callback`.

Wiadomość ta nie zawiera uzupełnionych znaczników czasu, na podstawie których najlepiej byłoby obliczać różnicę czasu δt pomiędzy kolejnymi wiadomościami. Z tego powodu różnicę czasu można obliczyć:

- **Opcja I** – wykorzystać bibliotekę `time` w Pythonie i obliczyć różnicę czasu pomiędzy kolejnymi odebranymi wiadomościami:

```
1 import time
2 t1 = time.time()
3 # ... coś się dzieje ...
4 t2 = time.time()
5 dt = t2 - t1 # różnica w sekundach (float)
```

- **Opcja II** – wersja uproszczona: sprawdzić, z jaką częstotliwością publikowana jest wiadomość na temacie `/joint_states`. Jest to polecenie używane w terminalu ROS, dlatego należy uruchomić je w nowym terminalu lub w *Jupyter Notebook* poprzedzając komendę znakiem `!`. Zalecane jest jednak użycie poniższego polecenia bezpośrednio w terminalu:

```
1 ros2 topic hz /joint_states
```

2. **Obliczyć wektor prędkości $\dot{\xi}_R$** robota na podstawie prędkości kół $\dot{\phi}$ oraz kątów skrętu θ_{kolo} (jeśli występują):

$$\dot{\xi}_R = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{B} \dot{\phi} \quad (9.2)$$

3. **Obliczyć pozycję (x, y) oraz orientację θ_{robot}** robota na płaszczyźnie na podstawie wyznaczonych prędkości z poprzedniego punktu.

Należy pamiętać, że występują tutaj dwa układy współrzędnych: globalny (związany z płaszczyzną) oraz lokalny (związany z robotem).

Następnie należy zaktualizować pola klasy:

```
1 self.x = 0 # Pozycja robota w osi X (metry)
2 self.y = 0 # Pozycja robota w osi Y (metry)
3 self.theta = 0 # Orientacja robota (kąt obrotu wokół osi Z, radiany)
```

4. **Uzupełnić wiadomość `odom_msg`**, wykorzystując wyznaczone wcześniej wartości pozycji robota w osiach X , Y oraz jego orientację θ_{robot} .
5. **Zweryfikować poprawność wyników na rzeczywistym robocie.** Zaproponować testy umożliwiające ocenę działania i dokładności odometrii. Sprawdzenie można przeprowadzić np. w *RViz* — po przejechaniu przez robota kwadratu w rzeczywistości trajektoria odometrii powinna przypominać kwadrat, a nie linię.

Zebrać dane do sprawozdania.

Ćwiczenie 4 – Analiza manewrowalności

Celem ćwiczenia jest wyznaczenie manewrowalności robota mobilnego. Wyznaczana jest ona na podstawie wzoru 8.3.

Aby możliwe było określenie manewrowalności należy wyznaczyć:

- stopień mobilności (δ_m) ze wzoru 8.1
- stopień sterowalności (δ_s) ze wzoru 8.2

Ćwiczenie 5 – Sprawozdanie

Sprawozdanie powinno zawierać następujące rozdziały:

1. „Rozwiązania ćwiczeń – kod programu”

Screeny z napisanym kodem programu. Należy pominąć schematy, importy, opisy i inne fragmenty pliku, które zostały dostarczone do laboratorium. Dla każdego ćwiczenia należy przygotować **oddzielny podrozdział** zawierający tylko kod programu.

Najważniejsza jest wyłącznie praca własna. Dodatkowe opisy do kodu nie są wymagane. Screeny mają na celu umożliwienie odczytania kodu bez konieczności uruchamiania środowiska w celu otwarcia pliku Jupyter Notebook.

2. „Testy kinematyki prostej”

Rozdział powinien zawierać ogólny opis przeprowadzonych testów kinematyki prostej. Wystarczająca jest obserwacja zachowania robota – nie są wymagane szczegółowe pomiary ani analizy.

3. „Testy odometrii”

Rozdział dotyczący testów potwierdzających poprawność działania odometrii. Należy go uwzględnić, jeśli ćwiczenie zostało wykonane.

4. „Problemy i trudności”

Rozdział opisujący napotkane problemy i trudności podczas realizacji ćwiczenia. Może również zawierać sugestie dotyczące sposobów rozwiązania pojawiających się problemów, w szczególności w przypadku występowania nieścisłości w instrukcji.